



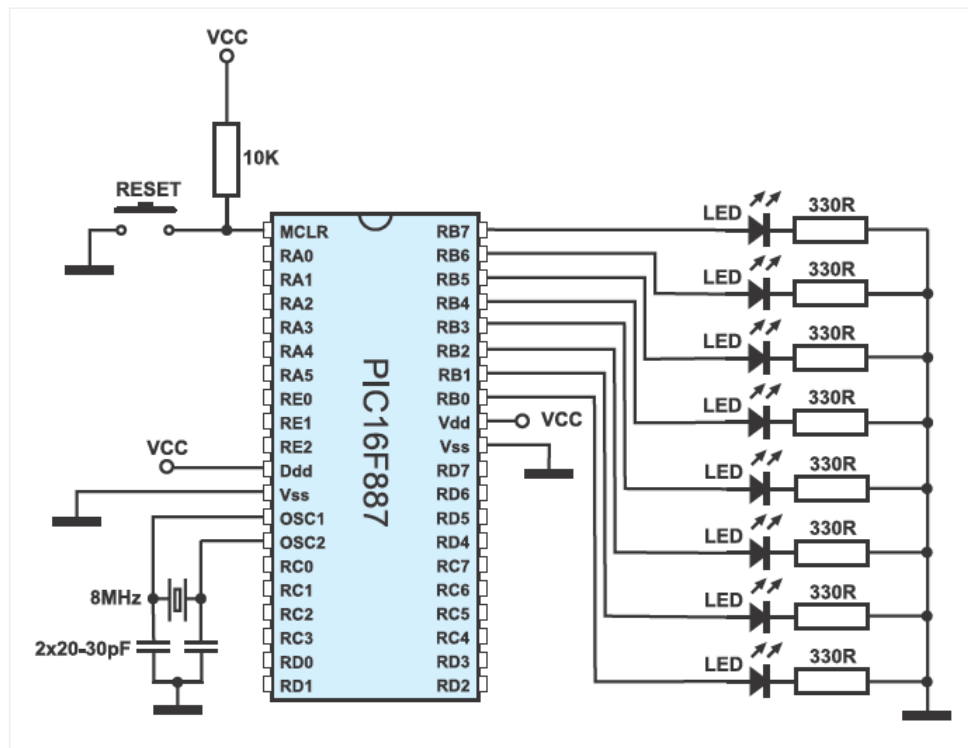
## 4.6 EXAMPLE 4

### MIKROELEKTRONIKA

Using timers TMR0, TMR1 and TMR2. Using interrupts, declaring new function...

If you have read the previous example, you probably have noticed a disadvantage of providing delays using loops. In all those cases, the microcontroller is 'captive' and does nothing. It simply waits for some time to pass. Such waste of time is an unacceptable luxury and some other method should be applied therefore.

Do you remember the story about timers? Interrupts? This example makes links between them in a practical way. The schematic is still the same as well as the challenge. It is necessary to provide a delay long enough to notice changes on a port. Timer TMR0 with assigned prescaler is used for this purpose. An interrupt is generated on every timer register overflow and every interrupt routine automatically increments the **cnt** variable by 1. When it's value reaches 400, the port B is incremented by 1. The whole procedure is performed 'behind the scenes', which enables the microcontroller to do something else.



```

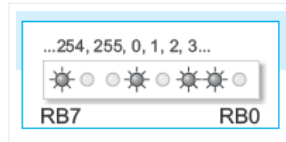
1  /*Header*****
2
3  unsigned cnt; // Define variable cnt
4
5  void interrupt() {
6      cnt++; // Interrupt causes cnt to be incremented by 1
7      TMR0 = 96; // Timer TMR0 is returned its initial value
8      INTCON = 0x20; // Bit T0IE is set, bit T0IF is cleared
9  }
10
11 void main() {
12     OPTION_REG = 0x84; // Prescaler is assigned to timer TMR0

```

```

13 ANSEL = 0; // All I/O pins are configured as digital
14 ANSELH = 0;
15 TRISB = 0; // All port B pins are configured as outputs
16 PORTB = 0x0; // Reset port B
17 TMR0 = 96; // Timer T0 counts from 96 to 255
18 INTCON = 0xA0; // Enable interrupt TMR0
19 cnt = 0; // Variable cnt is assigned a 0
20
21 do { // Endless loop
22   if (cnt == 400) { // Increment port B after 400 interrupts
23     PORTB = PORTB++; // Increment number on port B by 1
24     cnt = 0; // Reset variable cnt
25   }
26 } while(1);
27 }

```

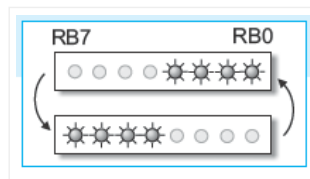


Interrupt occurs on every timer register TMR0 overflow.

```

1 /*Header******/
2
3 unsigned short cnt; // Define variable cnt
4
5 void interrupt() {
6   cnt++; // Interrupt causes cnt to be incremented by 1
7   PIR1.TMR1IF = 0; // Reset bit TMR1IF
8   TMR1H = 0x80; // TMR1H and TMR1L timer registers are returned
9   TMR1L = 0x00; // their initial values
10 }
11
12 void main() {
13   ANSEL = 0; // All I/O pins are configured as digital
14   ANSELH = 0;
15   PORTB = 0xF0; // Initial value of port B bits
16   TRISB = 0; // Port B pins are configured as outputs
17   T1CON = 1; // Set timer TMR1
18   PIR1.TMR1IF = 0; // Reset bit TMR1IF
19   TMR1H = 0x80; // Set initial value for timer TMR1
20   TMR1L = 0x00;
21   PIE1.TMR1IE = 1; // Enable interrupt on overflow
22   cnt = 0; // Reset variable cnt
23   INTCON = 0xC0; // Enable interrupt (bits GIE and PEIE)
24
25   do { // Endless loop
26     if (cnt == 76) { // Change port B state after 76 interrupts
27       PORTB = ~PORTB; // Number in port B is inverted
28       cnt = 0; // Reset variable cnt
29     }
30   } while (1);
31 }

```



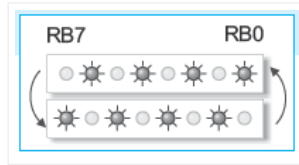
In this case, an interrupt is enabled after the timer register TMR1 (TMR1H, TMR1L) overflow occurs. Combination of bits changing on port B is different from that in the previous example.

```

1 /*Header******/
2
3 unsigned short cnt; // Define variable cnt
4
5 void Replace() {
6   PORTB = ~PORTB; // Define new function 'Replace'
7 } // Function inverts port state
8
9 void interrupt() {
10  if (PIR1.TMR2IF) { // If bit TMR2IF = 1,
11    cnt++; // Increment variable cnt by 1
12    PIR1.TMR2IF = 0; // Reset bit and
13    TMR2 = 0; // reset register TMR2
14  }
15 }
16
17 // main
18 void main() {
19   cnt = 0; // Reset variable cnt
20   ANSEL = 0; // All I/O pins are configured as digital
21   ANSELH = 0;
22   PORTB = 0b10101010; // Logic state on port B pins
23   TRISB = 0; // All port B pins are configured as outputs
24   T2CON = 0xFF; // Set timer T2
25   TMR2 = 0; // Initial value of timer register TMR2
26   PIE1.TMR2IE = 1; // Enable interrupt

```

```
27 INTCON = 0xC0; // Set bits GIE and PEIE
28
29 while (1) { // Endless loop
30     if (cnt > 30) { // Change PORTB after more than 30 interrupts
31         Replace(); // Function Replace inverts the port B state
32         cnt = 0; // Reset variable cnt
33     }
34 }
35 }
```



This time, an interrupt occurs after timer register TMR2 overflow occurs. The *Replace* function, which normally doesn't belong to C, is used in this example to invert port pins state.



4.6 Example 4 by MikroElektronika is licensed under a Creative Commons Attribution 4.0 International License, except where otherwise noted.